Chapter 1

Introduction

Chapter 1. Introduction

1.1 What is a constraint satisfaction problem?

1.2 Formal Definition of the CSP

1.3 Constraint Representation and Binary CSPs

1.5 Examples and Applications of CSPs

1.6 Constraint Programming

Almost everybody who works in artificial intelligence should know something about the **Constraint Satisfaction Problem** (CSP). CSPs appear in many areas, for instance, vision, resource allocation in scheduling and temporal reasoning.

The CSP is worth studying in isolation because it is a general problem that has unique features which can be exploited to arrive at solutions. The main objective of this course is to identify these properties and explain techniques for tackling CSPs.

We shall first define the standard CSP and the important concepts around it. To avoid ambiguity, concepts are defined both verbally and in first order predicate calculus (FOPC). However, the verbal definitions alone should be sufficient.

1.1 What is a constraint satisfaction problem?

Basically, a **CSP** is a problem composed of

- a finite set of **variables**, each of which is associated with
- a finite **domain**, and
- a set of **constraints** restricting the values variables can simultaneously take.

The task is to assign a value to each variable **satisfying all the constraints**.

Examples:

- The N-queens problem
- The car sequencing problem

1.1.1. Example 1 —The *N*-queens problem

The N-queens problem is a well known puzzle among computer scientists. Although the N-queens problem has very specific features (explained below) which can be exploited in solving it, it has been used extensively for illustrating CSP solving algorithms.

Given any integer N, the problem is to place N queens on N distinct squares in an $N \times N$ chess board, satisfying the constraint that no two queens should threaten each other. The rule is such that a queen can threaten any other pieces on the same row, column or diagonal. Figure 1.1 shows one possible solution to the 8-queens problem.

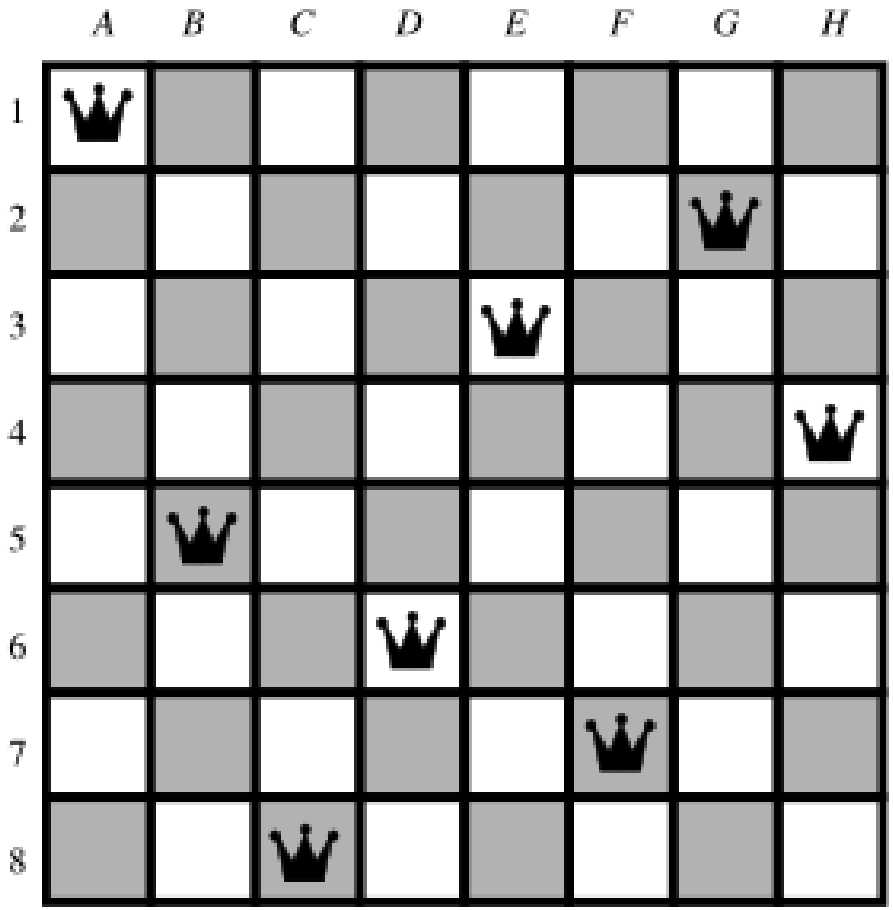


Figure 1.1: A possible solution to the 8-queens problem.

One way to formalize the 8-queens problem as a CSP is to see it as a problem with eight variables (i.e. a finite set of variables), each of which may take a value from A to H. The task is to assign values to the variables satisfying the above-specified constraints.

Other ways to formalize the 8-queens problem as a CSP?

1.1.2. Example 2 — The car sequencing problem

In modern car production, cars are placed on conveyor belts which move through different work areas. Each of these work areas specializes to do a particular job, such as fitting sunroofs, car radios or air-conditioners.

When a car enters a work area, a team of engineers in that area travels with the car while working on it. The production line is designed so as to allow enough time for the engineers to finish this job while the car is in their work area.

For example, if the time taken to install a sunroof is 20 minutes, and one car enters the conveyor belt every four minutes, then the work area for sunroof installation will be given a capacity of carrying (20 ÷ 4 =) five cars.

Figure 1.2 shows a section of the production line.

A production line is normally required to produce cars of different models. The number of cars required for each model is called the *production requirement*. Since cars of different models require different options to be fitted, not every car requires work to be done in every work area.

For example, one model may need air-conditioning and power brakes to be installed, but not a sunroof. The upper half of Figure 1.2 shows an example of the production requirement and the options required by four models. For example, 30 cars of model A are required, each of which needs a radio cassette, air-conditioning and power brakes to be fitted, but not a sunroof or anti-rust treatment.

Each work area is constrained by its resource constraint.

For example, if three teams of engineers are designated to fitting sunroofs, and the sunroof work area has a space capacity for five cars, then the sunroof work

area can cope with no more than three out of five cars requiring the fitting of sunroofs in any sub-sequence of cars on the conveyor belt. If more than three cars in any sequence of five cars require sunroofs, then the engineers would not have time to finish before the conveyor belt takes the cars away. The ratio 3/5 is called the *capacity constraint* of the work area for sunroof. In the example in Figure 1.2, the capacity constraints of the sunroof and radio cassette work areas are 3/5 and 2/3, respectively. We have not specified the capacity constraints of the other options there.

A *car-sequencing problem* is specified by the production and option requirements and the capacity constraints. Given the production requirements, the scheduler's task is to order the cars in the conveyor belt so that the capacity constraint of all the work areas are satisfied. In the above example, 120 cars of the four specified models must be scheduled. The sub-sequence shown in Figure 1.2 is:

..., *B, C, A, A, B, C, D, B, D, C*, ...

Production Requirements:

Options (✓ = required, × = not):	Model A	Model B	Model C	Model D	Total:
Sunroof	×	✓	✓	×	
Radio cassette	✓	×	✓	✓	
Air-conditioning	✓	✓	×	✓	
Anti-rust treatment	×	✓	✓	✓	
Power brakes	✓	×	✓	×	
Number of cars required:	30	30	20	40	120

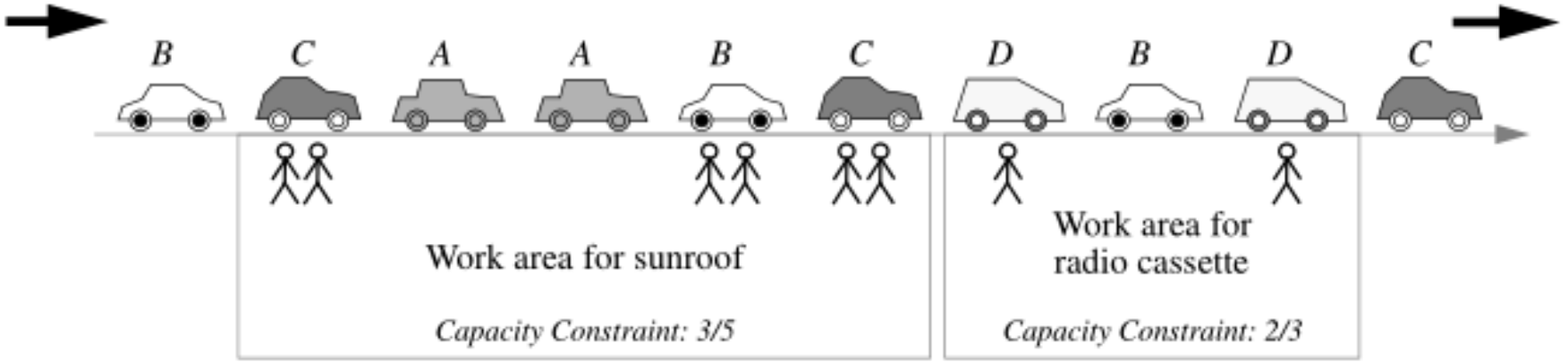


Figure 1.2. Example of a car sequencing problem

To check that it satisfies the capacity constraint of the sunroof work area, one has to look at every sub-sequence of five cars, e.g.

B, C, A, A, B
C, A, A, B, C
A, A, B, C, D
.....

Careful examination should convince readers that the sub-sequence (*C*, *A*, *A*) in Figure 1.2 actually violates the capacity constraints of the radio cassette work area.

The car-sequencing problem is difficult when a large number (say, hundreds) of cars are to be scheduled. Failure has been reported in attempting to solve it using theorem provers and expert system tools. It has been shown that this problem can be solved efficiently by formulating this problem as a CSP and applying CSP solving techniques to it.

We have already explained that a CSP is composed of variables, domains and constraints.

The car-sequencing problem can be formulated as a CSP in the following way.

One variable is used to represent the car model of one position in the conveyor belt (i.e. if there are *n* cars to be scheduled, the problem consists of *n* variables).

The domain of each variable is the set of car models, A to D in the above example.

The task is to assign a value (a car model) to each variable (a position in the conveyor belt), satisfying both the production requirements and capacity constraints.

1.2. Formal Definition of the CSP

Here we shall give a more formal definition of the CSP. Before doing so, we first define *domains*, *assignments* (which we call *labels* below), and the concept of *satisfying* in terms of set relations.

1.2.1. Definitions of domain and labels

Definition 1-1:

The **domain of a variable** is a set of all possible values that can be assigned to the variable. If x is a variable, then we use Dx to denote the domain of it. ■

When the domain contains numbers only, the variables are called **numerical variables**. The domain of a numerical variable may be further restricted to integers, rational numbers or real numbers. For example, the domain of an

integer variable is an infinite set {1, 2, 3, ...}. The majority of this course focuses on CSPs with finite domains.

When the domain contains boolean values only, the variables are called **boolean variables**. When the domain contains an enumerated type of objects, the variables are called **symbolic variables**. For example, a variable that represents a day of the week is a symbolic variable of which the domain is the finite set {Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday}.

Definition 1-2:

A **label** is a variable-value pair that represents the assignment of the value to the variable. We use $<x, v>$ to denote the label of assigning the value v to the variable x. $<x, v>$ is only meaningful if v is in the domain of x (i.e. $v \in Dx$).

■

Definition 1-3:

A **compound label** is the simultaneous assignment of values to a (possibly empty) set of variables. We use (<*x1*,*v1*><*x2*,*v2*>...<*xn*,*vn*>) to denote the compound label of assigning *v1*, *v2*, ..., *vn* to *x1*, *x2*, ..., *xn* respectively. ■

Since a compound label is seen as a set, the ordering of the labels in our representation is insignificant. In other words, (<*x*,*a*><*y*,*b*><*z*,*c*>) is treated as exactly the same compound label as (<*y*,*b*><*x*,*a*><*z*,*c*>), (<*z*,*c*><*x*,*a*><*y*,*b*>), etc. Besides, it is important to remember that a set does not have duplicate objects.

Definition 1-4:

A ***k*-compound label** is a compound label which assigns *k* values to *k* variables simultaneously. ■

Definition 1-5:

If m and n are integers such that $m \leq n$, then a **projection** of an n-compound label N to an m-compound label M, written as *projection(N, M)*, (read as: M is a projection of N) if the labels in M all appear in N.

$\forall$ (<*x1,v1*>...<*xm,vm*>), (<*z1,w1*>...<*zn,wn*>): {*x1*, ..., *xm*} $\subseteq$ {*z1*, ..., *zn*} :
projection((<*z1,w1*>...<*zn,wn*>),(<*x1,v1*>...<*xm,vm*>)) $\equiv$
<*x1,v1*>, ..., <*xm,vm*> $\in$ {<*z1,w1*>, ..., <*zn,wn*>} ■

We use this notation to indicate that projection((<*z1,w1*>...<*zn,wn*>), (<*x1,v1*>...<*xm,vm*>)) is only defined if (<*z1,w1*>...<*zn,wn*>) and (<*x1,v1*>...<*xm,vm*>) are compound labels, and {*x1*, ..., *xm*} is a subset of {*z1*, ..., *zn*}. It is undefined otherwise.

For example, (<*a*,1><*c*,3>) is a projection of (<*a*,1><*b*,2><*c*,3>), which means the proposition *projection((<a,1><b,2><c,3>), (<a,1><c,3>))* is true.

Definition 1-6:

The **variables of a compound label** is the set of all variables which appear in that compound label:

variables_of((<*x1,v1*> <*x2,v2*> ... <*xk,vk*>)) ≡ { *x1, x2, ..., xk* } ■

1.2.2. Definitions of constraints

A constraint on a set of variables is a restriction on the values that they can take simultaneously.

Conceptually, a constraint can be seen as a set that contains all the legal compound labels for the subject variables; though in practice, constraints can be represented in many other ways, which we shall discuss later:

- functions,
- inequalities,
- matrices, etc.,

Definition 1-7:

A **constraint** on a set of variables is conceptually a set of compound labels for the subject variables. For convenience, we use *CS* to denote the constraint on the set of variables *S*. ■

Definition 1-8:

The **variables of a constraint** is the variables of the members of the constraint:

variables_of(*Cx1 x2 … xk*) ≡ { *x1, x2, ..., xk* } ■

"Subsumed-by" is a binary relationship on constraints.

Definition 1-9:

If m and n are integers such that $m \leq n$, then an m-constraint M is **subsumed by** an n-constraint N (written as *subsumed-by(M,N))* if for all elements c in M there exists an element d in N such that c is a projection of d:

$\forall\ CM, CN: |M| = m \wedge |N| = n \wedge m \leq n:$
  subsumed-by(CM, CN) ≡
    ($\forall$ (<$x1$,$v1$>...<xm,vm>) $\in$ CM:
      ($\exists$ (<$z1$,$w1$>...<zn,wn>) $\in$ CN:
        projection((<$z1$,$w1$>...<zn,wn>),(<$x1$,$v1$>...<xm,vm>)))) ■

Here $|M|$ and $|N|$ denote the number of variables in M and N respectively. If:

$CM = \{(\langle a,1\rangle\langle c,3\rangle), (\langle a,4\rangle\langle c,6\rangle)\}$

and $CN = \{(\langle a,1\rangle\langle b,2\rangle\langle c,3\rangle), (\langle a,1\rangle\langle b,4\rangle\langle c,3\rangle), (\langle a,4\rangle\langle b,5\rangle\langle c,6\rangle)\}$,

then the proposition subsumed-by(CM, CN) is true.

In other words, if constraint M is subsumed by constraint N, then N is at least as restrictive as M. Apart from constraining the variables of M, N could possibly constrain other variables too (in the above example, CM constrains variables a and c, while CN constrains a, b and c).

1.2.3. Definitions of satisfiability

Satisfies is a binary relationship between a label or a compound label and a constraint.

Definition 1-10a:

If the variables of the compound label *X* are the same as those variables of the elements of the compound labels in constraint *C*, then *X* **satisfies** *C* if and only if *X* is an element of *C*:

satisfies((<*x1,v1*><*x2,v2*>...<*xk,vk*>) , *Cx1 x2 … xk*) ≡
(<*x1,v1*><*x2,v2*>...<*xk,vk*>) ∈ *Cx1 x2 … xk* ■

For convenience, *satisfies* is also defined between labels and unary constraints.

Definition 1-10b:

satisfies(<*x*,*v*>, *Cx*) ≡ (<*x*,*v*>) ∈ *Cx* ■

This allows us to write something like *satisfies(<x,v>, Cx)* as well as *satisfies((<x,v>),Cx)*. The concept of *satisfies(L,C)* is extended to the case when C is a constraint on a subset of the variables of the compound label L.

Definition 1-11:

Given a compound label *L* and a constraint *C* such that the variables of *C* are a subset of the variables of *L*, the **compound label** *L* **satisfies constraint** *C* if and only if the projection of *L* onto the variables of *C* is an element of *C*:

∀ *x1, x2, ..., xk*: ∀ *v1* ∈ *Dx1* , *v2* ∈ *Dx2* , ... , *vk* ∈ *Dxk* :
(∀ *S* ⊆ {*x1, x2, ..., xk*}:
satisfies((<*x1,v1*> <*x2,v2* >... <*xk,vk*>), *CS*) ≡
(∃ *cl* ∈ *CS* : projection((<*x1,v1*> ... <*xk,vk*>), *cl*))) ■

In other words, when we say that *L* satisfies *C*, we mean that if *C* is a constraint on the variables {*x1*, *x2*, ..., *xk*} or its subset, then the labels for those variables in *L* are legal as far as *C* is concerned.

For example, (<*a*,1> <*b*,2> <*c*,3> <*d*,4>) satisfies the constraint *Cc,d* if and only if (<*c*,3> <*d*,4>) is a member of *Cc,d*: *Cc,d* = {..., (<*c*,3> <*d*,4>), ...}.

1.2.4. Formal definition of constraint satisfaction problems

We stated earlier that a CSP is a problem with a finite set of variables, each associated to a finite domain, and a set of constraints which restrict the values that these variables can simultaneously take.

Definition 1-12:

A **constraint satisfaction problem** is a triple **(*Z*, *D*, *C*)**

where Z = a finite set of **variables** $\{x1, x2, ..., xn\}$;

D = a function which maps every variable in Z to a set of objects of arbitrary type: D: $Z \rightarrow$ finite set of objects (of any type)

We shall take Dxi as the set of objects mapped from xi by D. We call these objects possible **values** of xi and the set Dxi the **domain** of xi;

C = a finite (possibly empty) set of **constraints** on an arbitrary subset of variables in Z. In other words, C is a set of sets of compound labels.

We use *csp(P)* to denote that P is a constraint satisfaction problem. ■

Cx1 x2 … xk restricts the set of compound labels that *x1*, *x2*, ..., and *xk* can take simultaneously. For example, if the variable *x* can only take the values *a*, *b* and *c*, then we write *Cx* = {(<*x*,*a*>),(<*x*,*b*>),(<*x*,*c*>)}.

(Note the difference between *Cx* and *Dx*: *Cx* is a set of labels while *Dx* is a set of values.)

The value that *x* can take may be subject to constraints other than *Cx*. That means although <*x*,*a*> satisfies *Cx*, *a* may not be a valid value for *x* in the overall problem. To qualify as a valid label, <*x*,*a*> must satisfy all constraints which constrain *x*, including *Cx*,*y*, *Cw*,*x*,*z*, etc.

We focus on CSPs with finite number of variables and finite domains because, as illustrated later, efficient algorithms which exploit these features can be developed.

1.2.5. Task in a CSP

The task in a CSP is to assign a value to each variable such that all the constraints are satisfied simultaneously.

Definition 1-13:

A **solution tuple** of a CSP is a compound label for all those variables which satisfy all the constraints:

$$\forall \text{ csp}((Z, D, C)): \forall\, x1, x2, ..., xn \in Z: (\forall\, v1 \in Dx1\ , v2 \in Dx2\ , ..., vn \in Dxn\ :$$
$$\text{solution_tuple}(\ (\langle x1,v1\rangle \langle x2,v2\rangle \ ... \ \langle xn,vn\rangle), (Z, D, C)\ ) \equiv$$
$$((Z = \{x1, x2, ..., xn\}) \wedge$$
$$(\forall c \in C: \text{satisfies}((\langle x1,v1\rangle \langle x2,v2\rangle \ ... \ \langle xn,vn\rangle), c))) \blacksquare$$

A CSP is *satisfiable* if solution tuple exists. Depending on the requirements of an application, CSPs can be classified into the following categories:

(1) CSPs in which one has to find *any* solution tuple.
(2) CSPs in which one has to find *all* solution tuples.
(3) CSPs in which one has to find *optimal* solutions, where optimality is defined according to some domain knowledge. Optimal or near optimal solutions are often required in scheduling.

1.2.6. Remarks on the definition of CSPs

The CSP defined above is sometimes referred to as the *Finite Constraint Satisfaction Problem* or the *Consistent Labelling Problem*. The term "constraint satisfaction" is often used loosely to describe problems which need not conform to the above definition. In some problems, variables may have infinite domains (e.g. numerical variables). There are also problems in which the set of variables could change dynamically — depending on the value that one variable takes, different sets of new variables could emerge. Though these problems are important, they belong to another class of problems which demand a different set of specialized techniques for solving them.

1.3. Constraint Representation and Binary CSPs

We said earlier that if $S = \{x1, x2, ..., xk\}$, we use CS or $Cx1\ x2\ \ldots\ xk$ to denote the constraint on S. CS restricts the compound labels that the variables in S can simultaneously take.

A constraint can be represented in a number of different ways. Constraints on numerical variables can be represented by equations or inequalities; for example, a binary constraint Cx,y may be $x + y < 10$. A constraint may also be viewed as a function which maps every compound label on the subject variables to true or false.

Alternatively, a constraint may be seen as the set of all legal compound labels for the subject variables. This logical representation will be taken in this book as it helps to explain the concept of problem reduction — where *tightening a constraint* means removing elements from the set. This choice of representation should not affect the generality of our discussions.

One way in which to represent binary constraints is to use matrices of boolean values. For example, assume that variable x can take values 1, 2 and 3, and variable y can take values 4, 5, 6 and 7. The constraint on x and y which states that "$x + y$ *must be odd*" can be represented by a matrix, as shown in Figure 1.3.

C_{xy}		y			
		4	5	6	7
	1	1	0	1	0
x	2	0	1	0	1
	3	1	0	1	0

Figure 1.3. Matrix representing the constraint between x and y

The matrix in Figure 1.3 represents the fact that:

(<x,1><y,4>)
(<x,1><y,6>)
(<x,2><y,5>)
(<x,2><y,7>)
(<x,3><y,4>)
(<x,3><y,6>)

are all the compound labels that variables x and y can take.

Since a lot of research focuses on problems with unary and binary constraints only, we define the term *binary constraint problem* for future reference.

Definition 1-14:

A **binary CSP**, or **binary constraint problem**, is a CSP with unary and binary constraints only. A CSP with constraints not limited to unary and binary will be referred to as a **general CSP**. ■

It is worth pointing out that all problems can be transformed into binary constraint problems, though whether one would benefit from doing so is another question.

In general, if *x1*, *x2*, ..., *xk* are *k* variables, and there exists a *k*-ary constraint *CC* on them, then this constraint can be replaced by a new variable *W* and *k* binary constraints.

The domain of *W* is the set of all compound labels in *CC* (we mentioned earlier that we see constraints as sets of compound labels). Each of the *k* newly created binary constraints connects *W* and one of the *k* variables *x1* to *xk*.

The binary constraint which connects *W* and a variable *xi* requires *xi* to take a value which is projected from some values in *W*. This could be illustrated by the example in Figure 1.4. Let *x*, *y* and *z* be three variables in which the domains are

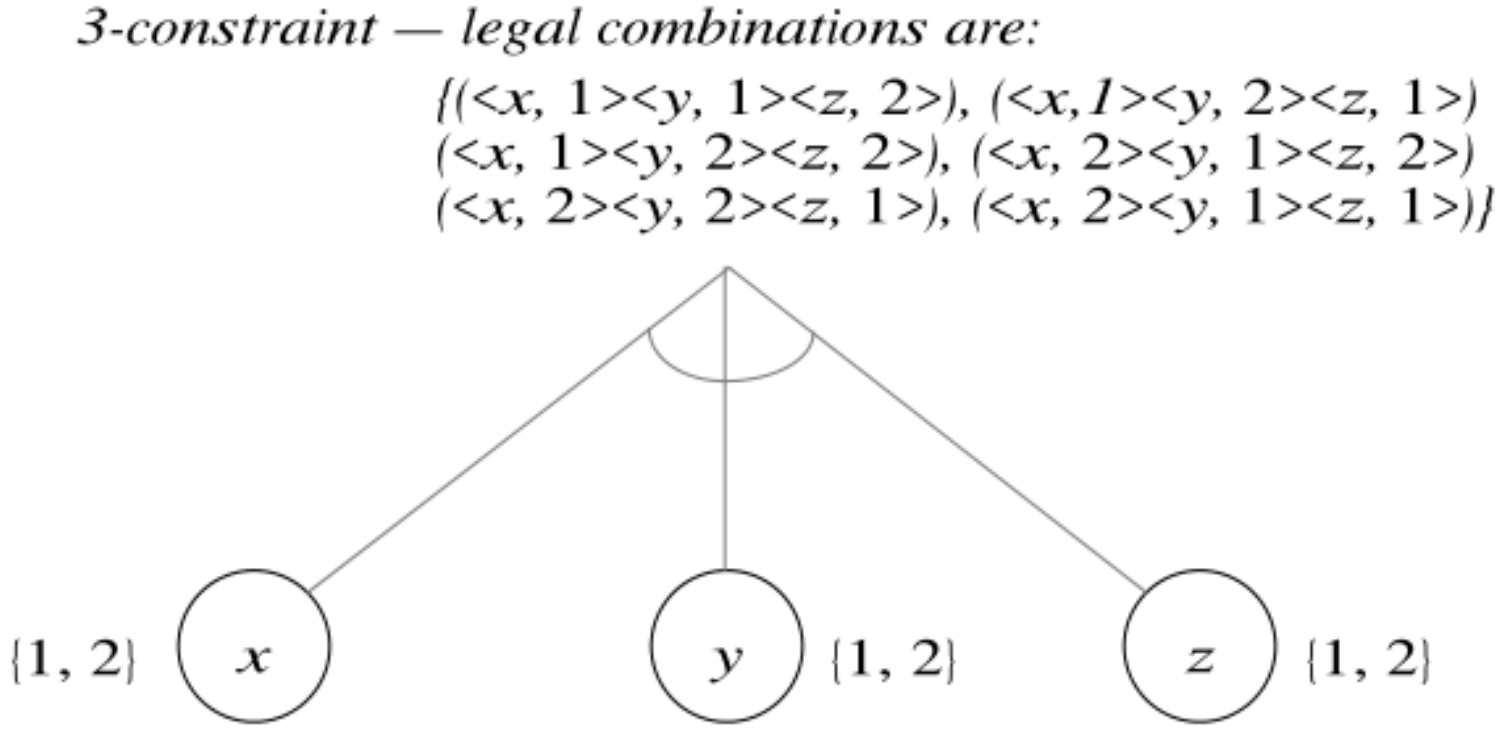
all {1, 2}, and there exists a 3-constraint insisting that not all three variables must take the same value (as shown in Figure 1.4(a)). This problem can be transformed into the binary constraint problem shown in Figure 1.4(b). In the transformed problem the variable *W* is created. The domain of *W* is the set of compound labels in *Cx,y,z*:

(<*x*,1>,<*y*,1>,<*z*,2>) (<*x*,1>,<*y*,2>,<*z*,2>)
(<*x*,1>,<*y*,2>,<*z*,1>) (<*x*,2>,<*y*,1>,<*z*,2>)
(<*x*,2>,<*y*,1>,<*z*,1>) (<*x*,2>,<*y*,2>,<*z*,1>)

The constraint between *W* and *x*, say, is that *projection(vW , (<x,vx>))* must hold, where *vW* and *vx* are the values that *W* and *x* take, respectively. For example, according to this constraint, if *W* takes the value (<*x*,1>,<*y*,1>,<*z*,2>), then *x* must take the value 1.

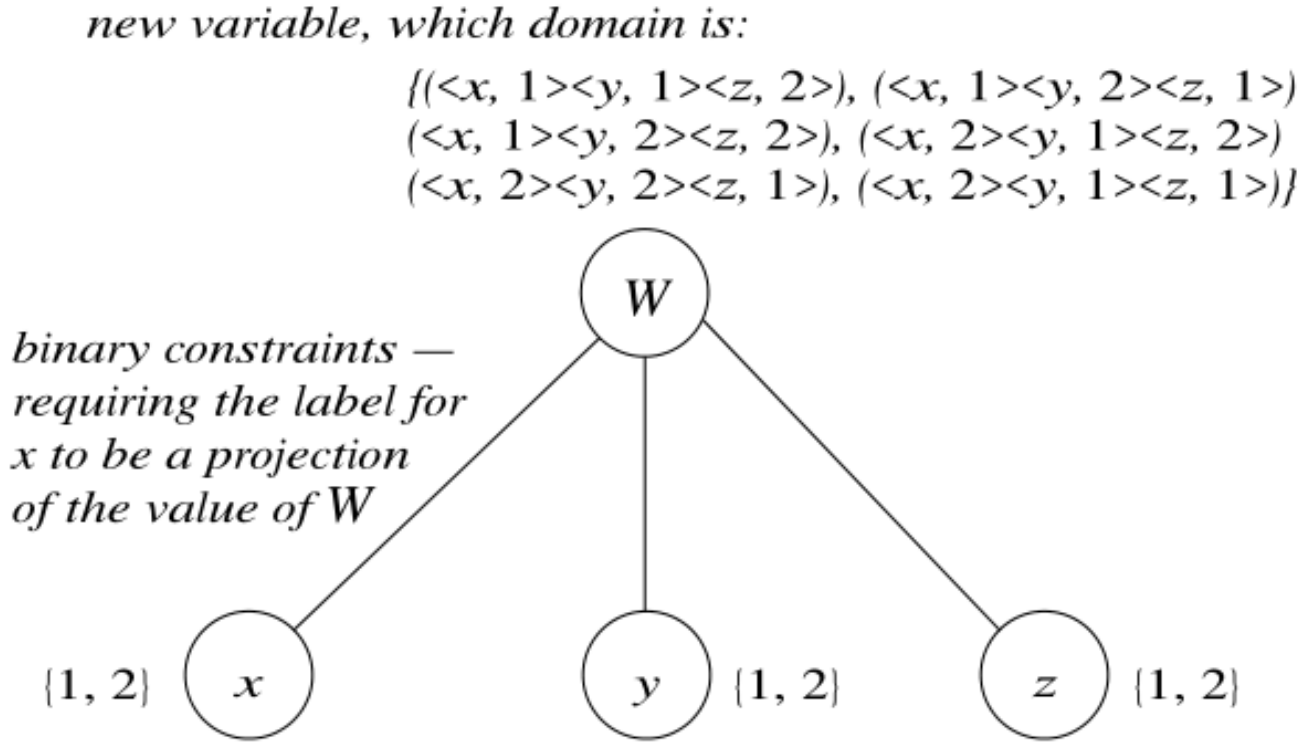
By removing k-ary constraints for all $k > 2$, we introduce new variables which have large domains. Whether one could benefit from this transformation depends on what we can do with the resulting problem. A number of CSP solving techniques which we shall illustrate in this book are applicable only to binary CSPs.

Later we shall explain that every CSP is associated to a *dual CSP*, which is also a binary CSP.



(a) A problem with the 3-constraint which disallows all of x, y and z to take the same values simultaneously. The domains of all x, y and z are {1, 2}

Figure 1.4 Transformation of a 3-constraint problem into a binary constraint



(b) A binary constraint problem which is transformed from (a). A new variable W is created, in which the domain is the set of all compound labels for x, y and z. The constraints between W and the other three variables require that **labels for** x, y and z must be projections of W's **value**

Figure 1.4 Transformation of a 3-constraint problem into a binary constraint

1.5. Examples and Applications of CSPs

To help understand what a CSP is and where they appear, we shall look at some examples and applications of CSPs in this section.

1.5.1. The *N*-queens problem

In this section, we shall formulate the *N*-queens problem that we introduced in Section 1.1.1 according to the formal definition of CSP, and illustrate that a problem can be formulated as a CSP in different ways.

1.5.1.1. Problem formalization

To formalize a problem as a CSP, we must identify a set of variables, a set of domains and a set of constraints. One way to formalize the 8-queens problem as a CSP is to make each of the eight rows in the 8-queens problem a variable: the set of variables $Z = \{Q1, Q2, ..., Q8\}$.

Each of these eight variables can take one of the eight columns as its value. If we label the columns with values 1 to 8 (for computation purposes, which will be made clear below), then the domains of all the variables in this CSP are as

follows: $DQ1 = DQ2 = ... = DQ8 = \{1, 2, 3, 4, 5, 6, 7, 8\}$.

Now let us look at the set of constraints. The fact that we represent each row as a variable has ensured that no two queens can be on the same row. To make sure that no two queens are on the same column, we have the following constraint:

Constraint (1): $\forall\ i, j: Qi \neq Qj$

To make sure that no two queens are on the same diagonal, we can include the following constraint in our set of constraints:

Constraint (2): $\forall\ i, j$, if $Qi = a$ and $Qj = b$, then $i - j \neq a - b$, and $i - j \neq b - a$.

(Making the values integers allows us to do arithmetic with them.) To represent these constraints, we could explicitly record the set of all compatible values

between each pair of variables. Alternatively, we can make them functions or procedures — given a pair of labels, these functions or procedures return true or false, depending on whether the given labels are compatible or not. Program 1.1 is a simple example of such a piece of code.

```
/*compatible1( X/Vx, Y/Vy )
X/Vx represents the X-th row, Vx-th column; and Y/Vy repre-
    sents the Y-th row, Vy-th column (where Vx and Vy both
    range from 1 to 8), compatible1/2 succeeds if and only if X/
    Vx and Y/Vy are compatible according to the constraints in
    the N-queens problem.
*/
    compatible1( X/Vx, Y/Vy ) :-
    Vx =\= Vy,                          /* Constraint (1) */
    X - Y =\= Vx - Vy,                  /* Constraint (2) */
    X - Y =\= Vy - Vx.                  /* Constraint (2) */
```

Program 1.1: Functional representation of a constraint in the *N*-queens problem

Under this problem formalization, there are 88 combinations of values for the eight variables to be considered. In general, an N-queens problem has N^N candidate solutions to be considered.

1.5.1.2. Alternative formalization of the *N*-queens problem

It is worth pointing out here that there is often more than one way to formalize a problem as a CSP. The *N*-queens problem need not be formalized in the above way. An alternative representation is to use *Q1*, *Q2*, ..., *Q8* to represent the positions of the queen (rather than the column of each queen in the above formalization). If the 64 squares in the 8 × 8 board are numbered 1 to 64, then the domain of each variable becomes {1, 2, ..., 64}. In other words:

$Z = \{Q1, Q2, ..., Q8\}$
$DQ1 = DQ2 = ... = DQ8 = \{1, 2, 3, 4, ..., 64\}$.

Let us assume that we number the squares from left to right, top to bottom. Then given a number which represents a square, the row and column of that square can be computed as follows:

row = (number div 8) + 1 column = (number mod 8) + 1

Given the rows and columns of two squares, we can check whether they are compatible with each other using the codes shown in Program 1.2.

```
/*compatible2( N1, N2 )
given N1 and N2, which both range from 1 to 64, this predicate
   succeeds if and only if N1 and N2 are compatible according
   to the constraints in the 8-queens problem.
*/
compatible2( N1, N2 ) :-
    R1 is (N1 div 8) + 1, C1 is (N1 mod 8) + 1,
    R2 is (N2 div 8) + 1, C2 is (N2 mod 8) + 1,
    R1  =\= R2,
    C1  =\= C2,
    R1 - R2  =\= C1 - C2,
    R1 - R2  =\= C2 - C1.
```

Program 1.2: Alternative functional representation of a constraint in the 8-queens problem

Some formalizations of a problem are easier to solve than others. The 8-queens problem formalized in this section allows 648 combinations of 8-compound labels, which makes the problem potentially more difficult to solve than the CSP formalized in the preceding section (which allows only 88 combinations of 8-compound labels). The formalization in the preceding section in fact has built into it the constraint that no two queens can be placed in the same row.

1.5.1.3. Caution about benchmarking using the *N*-queens problem

The *N*-queens problem is used to illustrate a number of CSP solving algorithms, but it is worth pointing out that benchmarks on different algorithms produced using this problem must be interpreted with caution.

This is because the *N*-queens problem has very specific features: firstly, it is a binary constraints problem; secondly, every variable is constrained by every other variable, which need not be the case in other problems.

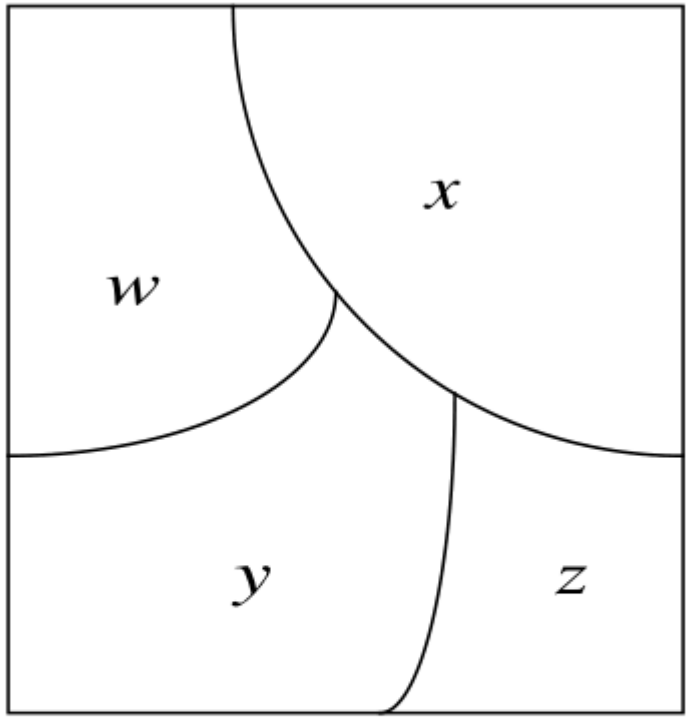
More importantly, in the *N*-queens problem, each label for every variable conflicts with at most three values of each other variable, regardless of the number of variables (i.e. N) in the problem.

For example, <1,2> has conflict with <2,1>, <2,2> and <2,3>. In an 8-queens problem, for example, when 2 is assigned to Queen 1, there are 5 out of 8 values that Queen 2 can take. But in the 1,000,000-queens problem, there are 999,997 out of 1,000,000 values that Queen 2 can take after <1,2> has been committed to.

Therefore, constraints get looser as N grows larger (see formal definition of tightness in Definition 2-13). Such features may not be shared by many other CSPs.

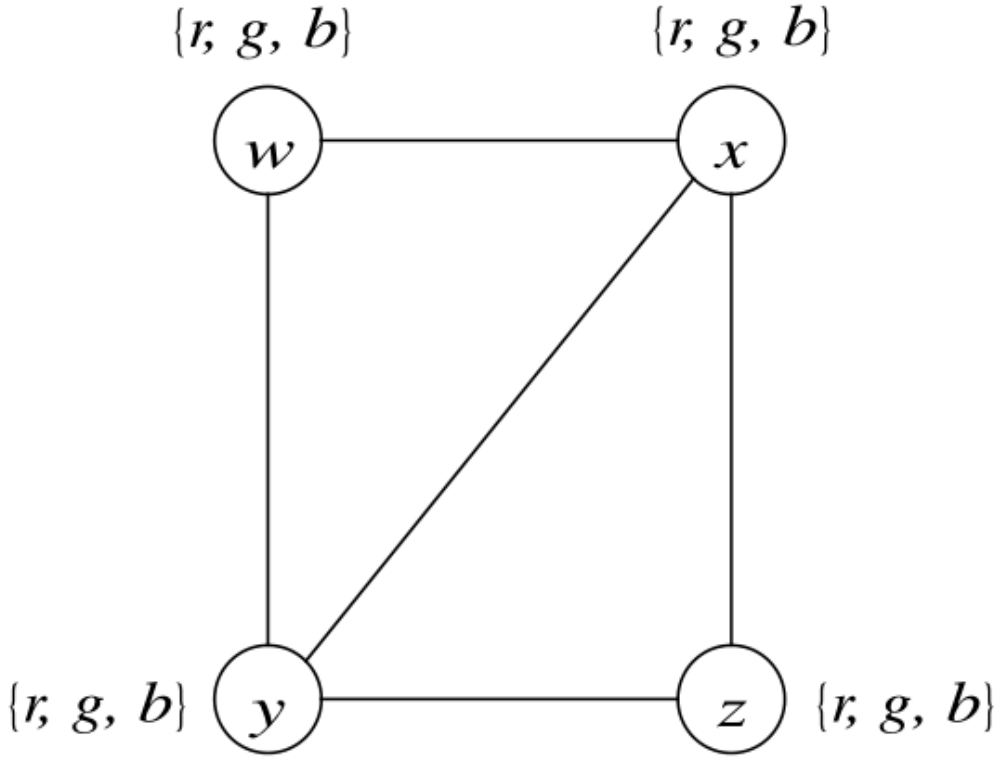
1.5.2. The graph colouring problem

Another problem which is often used to explain concepts and algorithms for the CSP is the *colouring problem*. Given a graph and a number of colours, the problem is to assign colours to those nodes satisfying the constraint that no adjacent nodes should have the same colour assigned to them. One instance of the colouring problem is the *map colouring problem*: the problem is to colour the different areas of a given map with a limited number of colours, subject to the constraint that no adjacent areas in the map have the same colour. Figure 1.5(a) shows an example of a map which is to be coloured. The map colouring problem is an instance of the general *graph colouring problem*, as a map can be represented by a graph where each node represents an area in the map, and every pair of nodes which represent two adjacent areas in the map is connected by an edge (see Figure 1.5(b)).



(a) map to be coloured

Figure 1.5 Example of a map colouring problem



(b) constraint graph of the CSP in (a): w, x, y and z are variables which all have the same domain $\{r, g, b\}$

Figure 1.5 Example of a map colouring problem

The areas to be coloured in Figure 1.5(a) are *w*, *x*, *y* and *z*. Assume that we are allowed to label the map with three colours only: *r* (for red), *g* (for green) and *b* (for blue). The values in {} next to the nodes (i.e. variables) in Figure 1.5(b) specify the domain. Each of the edges in the graph in Figure 1.5(b) represents a constraint which states that the connected nodes must not take the same value. The constraint on variables *A* and *B* (denoted *CA,B*) is conceptually seen as the set {(<*A*,*r*><*B*,*g*>), (<*A*,*r*><*B*,*b*>), (<*A*,*g*><*B*,*r*>), (<*A*,*g*><*B*,*b*>), (<*A*,*b*><*B*,*r*>), (<*A*,*b*><*B*,*g*>)}. (In practice, it can be represented by other means, e.g. a function). One solution tuple for this problem is: (<*A*,*r*> <*B*,*g*> <*C*,*b*> <*D*,*r*>). To summarize, the CSP (*Z*, *D*, *C*) for this problem is:

$Z = \{w, x, y, z\}$

$Dw = Dx = Dy = Dz = \{r, g, b\}$

$C = \{Cw,x, Cw,y, Cx,y, Cx,z, Cy,z\}$

1.5.3. The scene labelling problem

The *scene labelling problem* in computer vision is probably the first CSP to be formalized. In vision, the scenes are normally captured as images by cameras. After some preprocessing, lines can be recognized from the images, then scenes like the one shown in Figure 1.6 are generated.

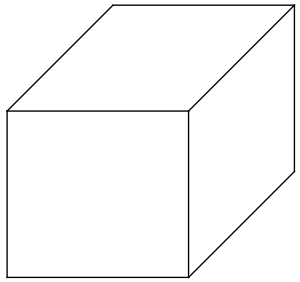


Figure 1.6 Example of a scene to be labelled

To recognize the objects in the scene, one must first interpret the lines in the drawings. One can categorize the lines in a scene into the following types:

(1) **convex edges**

A *convex edge* is an edge formed by two planes, both of which extend (from the edge) away from the viewer. Convex edges are marked by "+".

(2) **concave edges**

A *concave edge* is an edge formed by two planes both of which extend (from the edge) towards the viewer. Concave edges are marked by "−".

(3) **occluding edges**

An *occluding edge* is a convex edge where one of the planes is hidden behind the other and therefore not seen by the viewer. Occluding edges are marked by either "→" or "←", depending on the situation. If one moves along an occluding edge following the direction of the arrow, the area on the right represents the face of an object which can be seen by the viewer, and the area on the left represents the background or some faces of the objects at the back.

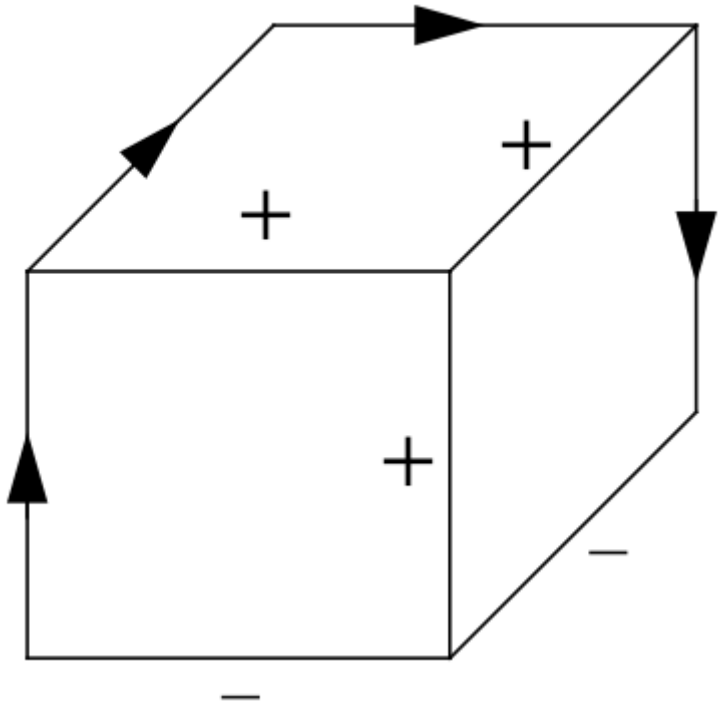


Figure 1.7 The scene in Figure 1.6 with labelled edges

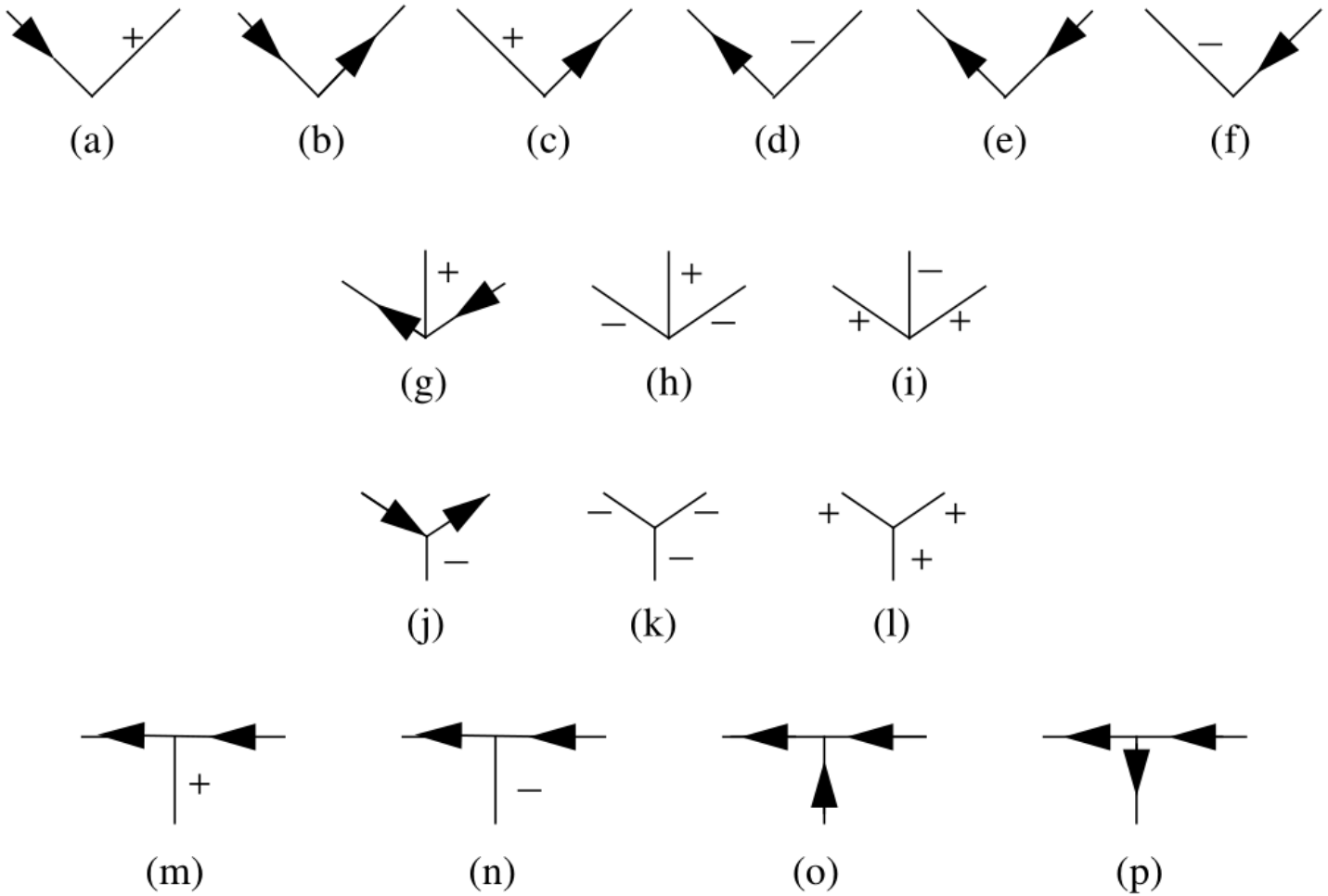


Figure 1.8 Legal labels for junctions (from Huffman, 1971)

The scene in Figure 1.6 can be labelled as shown in Figure 1.7. Given any junction independent of the scene, there are limited choices of labels. These choices are shown in Figure 1.8.

One way in which to formalize the scene labelling problem as a CSP is to use one variable to represent the value of a line in the scene. For example, in the scene in Figure 1.6 we have the following variables: $\{A, B, C, D, E, F, G, H, I\}$, as shown in Figure 1.9. The domain of each variable is therefore the set $\{+, -, \rightarrow, \leftarrow\}$.

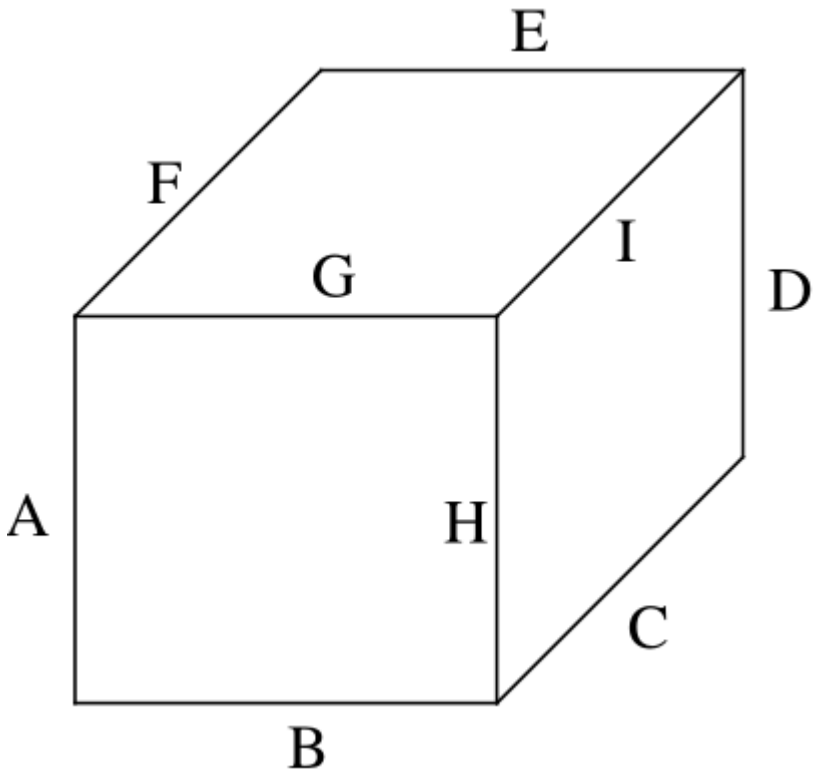
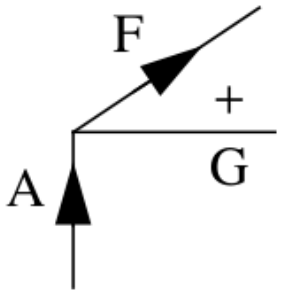
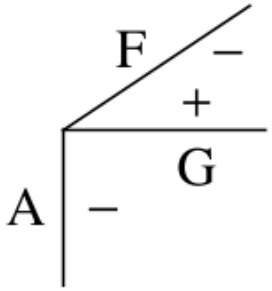
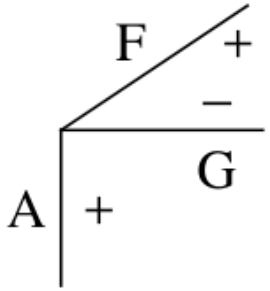


Figure 1.9 Variables in the scene labelling problem in Figure 1.6

The limited choices of label combinations in the junctions (as shown in Figure 1.8) impose constraints on the variables. Since lines *A*, *F*, *G* form an arrow, according to (g)-(i) in Figure 1.8, the values that these three variables can take simultaneously are restricted to:







(<A,→> <F,→> <G,+>) (<A,–> <F,–> <G,+>) (<A,+> <F,+> <G,–>)

Similarly, every other junction posts constraints to the labelling of the lines which form it. The task is to label all the variables, satisfying all the constraints. One may want to find one or all solutions to this problem, depending on the need of subsequent processing. Waltz introduced an algorithm, referred to as the *Waltz filtering algorithm*, for solving this problem. The algorithm is based on constraint propagation, and is discussed in Chapter 4.

1.5.4. Temporal reasoning

Temporal reasoning, which involves constraint satisfaction, is an important area in *AI planning* and many other applications (e.g. see Tsang, 1987b; Dechter *et al*. 1991). Events are all temporally related to each other. Depending on the time structure that one uses, different sets of temporal relations apply. In early research in planning, the world is simplified in such a way that all events are assumed to be instantaneous. In that case, three relations are possible between any two events *A* and *B*: "*A before B*", "*B before A*" or "*A equals B*". Allen [1983] points out that when durations in events are reasoned about, 13 relations are possible between any two events. These relations are shown in Figure 1.10.

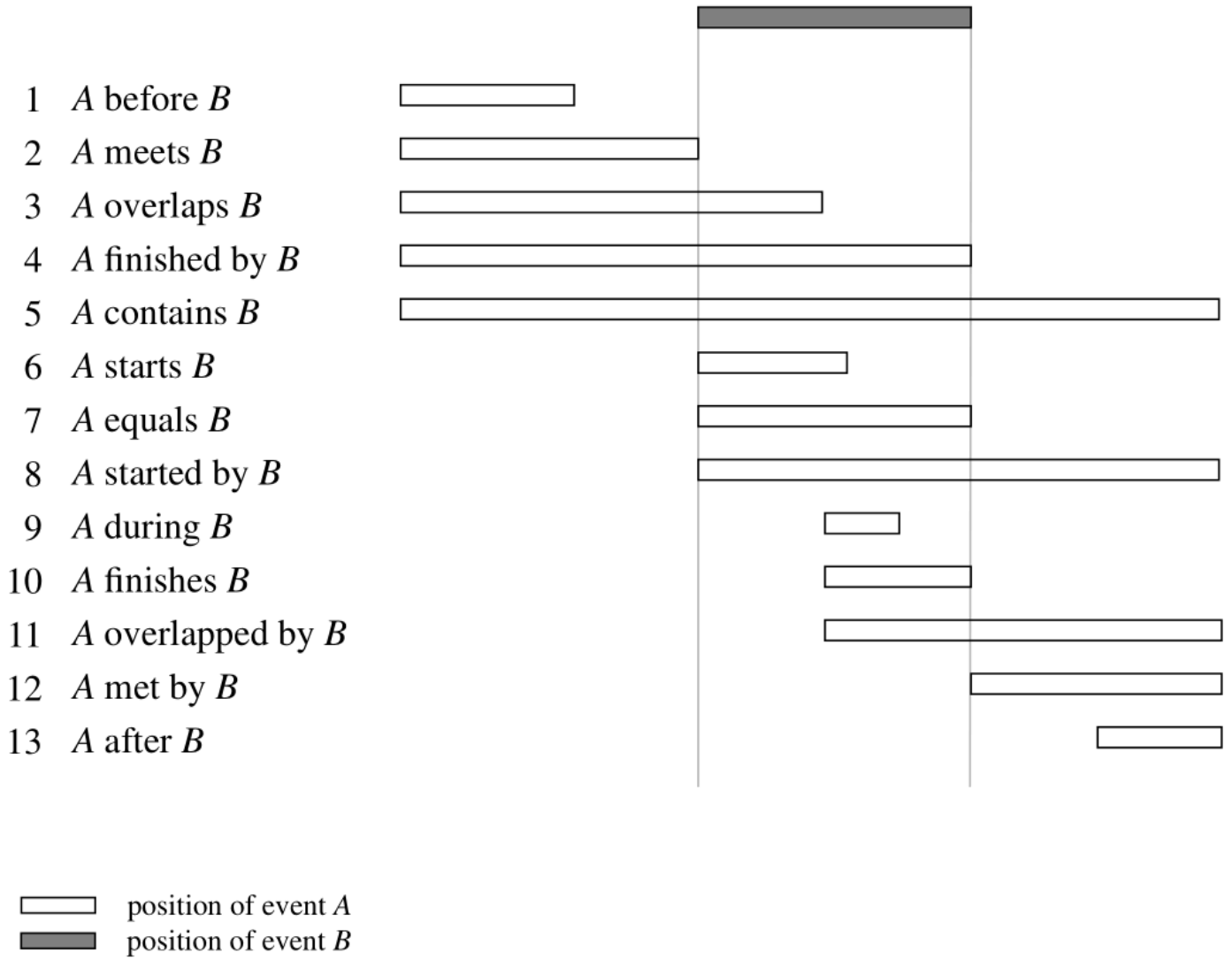


Figure 1.10 Thirteen possible temporal relations between two events

In planning and scheduling, one has to determine the temporal relationship between events. There are basically two approaches.

One is to assume one temporal relation per pair of events at a time, and backtrack when the hypothetical situation has been proved to be over-constrained. Most conventional planners do this. These planners adopt the assumption that events are instantaneous, therefore their ability to represent real life temporal knowledge is limited.

The other approach is to reason with all disjunctive temporal relations simultaneously. In order to schedule the events, one has to assign one temporal relation between each pair of events. Each CSP variable represents the temporal relationship between a pair or events. (Among n events, there are $n \times (n - 1) \div 2$ temporal relations, i.e. variables.) Each variable may take one of the 13 primitive relations in Figure 1.10 as its value. There are time constraints on the values of each variable.

For example, if A is before B, and B is before C, then A must be before C. If A overlaps B, and B overlaps C, then A must overlap, meet or be before C. The task is then to find a consistent set of primitive relations between the intervals — a set which satisfies all the constraints.

1.5.5. Resource allocation in AI planning and scheduling

Resource allocation and scheduling are better known applications of CSP. The car sequencing problem described in Section 1.1.2 is an example of a scheduling problem to which CSP solving techniques have been applied successfully. A typical scheduling problem is a problem in which one is given a set of jobs and asked to allocate resources to them. Each job may require a number of resources, which include time (during which these jobs are finished), machines, tools, manpower, etc.

Resource allocation, especially when time and shared resources are involved, is basically a CSP.

Each variable in the CSP represents one shared resource requirement. For example, variable X may represent the machine requirement of a job.

The domain of a variable is the set of possible values that this variable can take. The domain of *X* in the above example may be the set of machines available in the workshop which have the capacity to do the job, e.g. {machine-203, machine-208, machine-209}.

Assigning a value to a variable represents the allocation of a resource to a job.

The allocation of resources is normally constrained in many ways. For example, among the *M* machines available to a job *J*, only machines *P*, *Q* and *R* have the capacity to cope with job *J*. Very often, one machine can only process one job at a time. Sometimes, if job *J* is to use machine *M1*, then it must also be given certain tools and certain engineers.

The task is to allocate to each variable a value such that all the constraints are satisfied.

1.5.6. Graph matching

In semantic networks, one may want to check whether a particular concept is present. This problem can be seen as a graph matching problem, as defined below. Given two graphs *G1* and *G2*, the problem is to check whether *G2* has a subgraph which matches *G1*. Graph (*V1*, *E1*) contains graph (*V2*, *E2*) if:

(1) every node in *V2* can be mapped to a distinct node in *V1*; and
(2) for all *x1*, *y1* in *V1* and *x2*, *y2* in *V2*, if *x2* and *y2* are mapped to *x1* and *y1*, respectively, then whenever (*x2*,*y2*) is an edge in *E2*, then (*x1*,*y1*) is an edge in *E1*.

Figure 1.11 shows an example of a graph matching problem. Given the graphs *G1* and *G2* (shown in Figures 1.11(a) and (b), respectively), the task is to find out whether *G2* contains a subgraph of *G1*.

This can be formalized as a CSP where the variables are *A*, *B*, *C*, *D* and *E*, and the domains for all of them are {a,b,c,d,e,f,g,h,i,j}.

The constraint is that for all compound labels (<*x*,p><*y*,q>), if *x* and *y* are connected in *G1*, then p and q must be connected in *G2*.

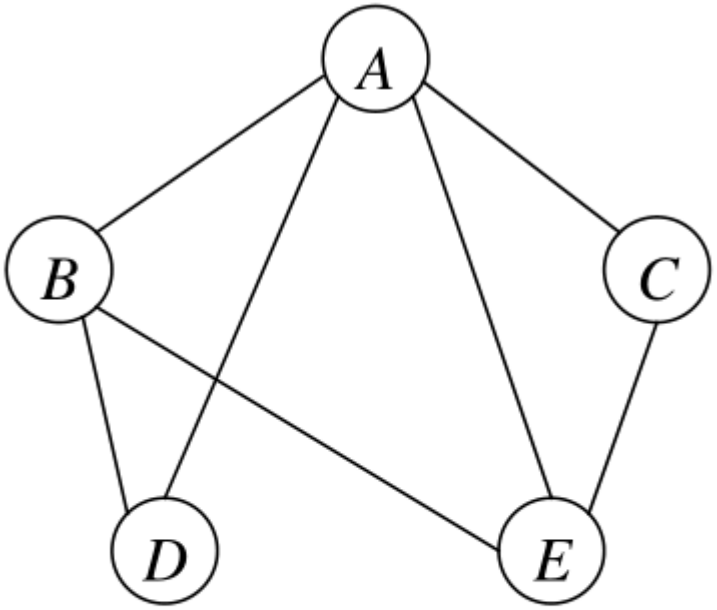
For example, (<*A*,h><*B*,g>) satisfies the constraint on *A* and *B* because (g,h) is an edge in *G2*. A little reflection should convince the readers that the compound label (<*A*,h><*B*,g><*C*,e><*D*,d><*E*,b>) is a solution tuple to this problem.

1.5.7. Other applications

In natural language parsing, each word has a finite number of roles that it can play. The language restricts the domain of roles (e.g. “noun”, “verb”, “adverb”, etc.) that each word can play. The grammar of the language restricts the roles that a string of words can take simultaneously. Part of the parsing task is to identify the roles of each word. Rich & Knight [1991] advocate that this task is a CSP.

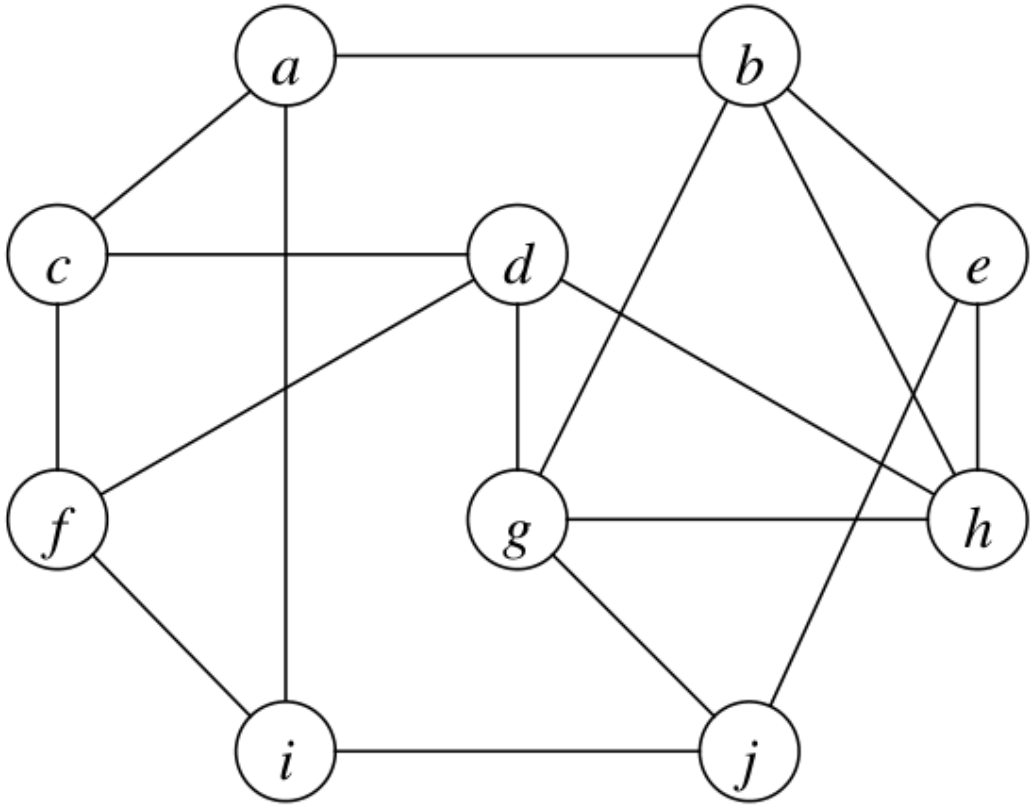
Database queries often have variables in them. Instantiating the variables in a database query is a CSP. Query optimization is an important database research area in which CSP solving techniques can be applied. On the other hand, techniques developed in query optimization research can be used in CSP solving. The tree-clustering method described in Chapter 8 is one example of such cross-fertilization of the two research disciplines.

CSP techniques have also been applied to parameter setting for greenhouses in agricultural applications, and demonstrated to be successful. Problem reduction techniques in CSP (see Chapters 2 to 4) have been demonstrated as being effective for cutting down search spaces for spatial reasoning.



(a) graph G_1

Figure 1.11 Example of a graph matching problem.
Does graph *G2* contain a subgraph which matches graph *G1*?



(b) graph G_2

Figure 1.11 Example of a graph matching problem.
(The answer is yes, if A→h, B→g, C→e, D→d, E→b)

1.6. Constraint Programming

The generality of the CSP has lead to the development of *constraint programming languages*. These languages provide built-in functions (or predicates) for describing commonly encountered constraints, and help users to solve problems by applying techniques which have been developed in CSP research.

Many approaches for constraint programming are based on and extended from the logic programming paradigm. Some of the better known constraint logic programming languages and systems are CLP, PROLOG III and CHIP. In these languages, unification in conventional logic programming is replaced by constraint satisfaction. Numerical constraints are being focused on. The idea is to hide the constraint solving techniques from the user.

One of the main objectives of developing CLP is to define a class of logic programming languages with well defined semantics under a particular equational theory. An instance of CLP called CLP(*R*) (*R* here stands for real numbers) has been implemented and proposed for applications such as electrical engineering and option trading. In CLP(*R*), constraints are handled incrementally using linear programming methods such as the simplex method.

Prolog III was developed with similar goals as CLP, but with refined manipulation on trees (including infinite trees), lists and boolean variables. It has been developed into a commercial product that has been demonstrated to be efficient and elegant in problem solving. Like CLP, Prolog III basically uses the simplex method for handling equations and inequalities with numerical variables.

CHIP is a logic programming language for handling symbolic, boolean as well as numerical variables. Search techniques, discussed later, are used to instantiate symbolic variables. The basic search strategy used in CHIP is called *forward checking* (FC). It is used together with a heuristic called the *fail first principle* (FFP). The combination of this search strategy and heuristic has been found to be very effective. CHIP has been applied to a number of problems, and success has been claimed. Some of the reported applications of CHIP include the car-sequencing problem, the spares allocation problem, job-shop scheduling, warehouse location, circuit verification (to verify that an implementation of a circuit meets its specifications) and the cut stock problem.

The success of CHIP has led to the development of two other commercially available languages, Charme and PECOS. The basic CSP solving techniques used in them are no different from CHIP, and therefore the comparison among CHIP, Charme and PECOS is down to the differences in their language types and their implementation efficiency.

Charme uses the syntax of C, and one of its merits is that it can easily be integrated into the users' other C programs. Arrays (which have to be implemented by lists in CHIP) are introduced in Charme. It has been applied to similar problems as CHIP. PECOS uses LISP syntax. Both Charme and PECOS are mainly built to handle symbolic but not numerical and boolean variables (boolean variables may be represented by symbolic variables with specific constraints such as logical AND and logical OR).